



Figure 2: App server login page

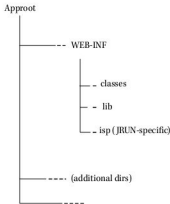


Figure 3: Web application directory structure

1. Model: The content or the actual data (or logic)
2. View: The presentation of data to the clients
3. Controller: It controls the application—for example, calling the model for business logic and then forwarding the request to 'view' for its presentation.

The Web application will be based on servlets and JSP. In very simple terms, servlets are server side applications that can dynamically extend the functionality of a Web server, and JSP is a presentation technology based on servlets. If you want to know more details about servlets and JSP then I'd recommend doing a Google search for the answers.

Typical Web application development on an app server requires a directory structure like in Figure 3. Here, *Approot* is the root directory for our Web application. It contains the default page that will be served by the server to its clients.

In addition, it contains a special directory named *WEB-INF*. This is the directory where our Web application-specific files will be stored. *WEB-INF* contains the following files and folders:

- The deployment descriptor file named *web.xml*.
- A *classes* folder that contains the model and controller classes of our MVC Web application
- A *lib* folder that contains third-party jar files
- A *jsp* folder that is JRUN specific; it will not be used in our sample application.

For our prime number generator Web application, create a directory structure as shown below:

- The top-level directory *Approot* contains the default page *index.html*, the *img* directory (which contains the images used in *index.html*), *result.jsp* (view of our MVC Web application), and the *WEB-INF* directory.
- *WEB-INF* contains *classes* and the *web.xml* file (a

deployment descriptor file used by the server to map servlet names to its classes).

There is no need of *lib* and *jsp* directories, as we are not using third-party *jar* files or *jsp*.

The default page for the prime number generating Web application, *index.html*, looks like what's shown below:

```

<html>
<!-- Some sample html -->
<head>

    <title>Prime number generator Web Service</title>

</head>
<body>

    <h1 align="center">Prime number generator Web Service</h1>
<center>

</center>
<center>

    <form method="GET" action="PrimeSearcher.do"
    enctype="multipart/form-data">

    <b>Enter the number till which you need prime numbers:</b>
    <input type="text" name="number" size="5"/><br/>
    </center>
    <br>
    <center>

    <input type="SUBMIT" value="GENERATE PRIMES"/>
</center>
<br>
<br>
<br>
</form>
</body>
</html>
  
```

This *index.html* is presented to clients who access the Web application. As you can see in the above snippet, the method is *GET* and the action servlet is *PrimeSearcher.do*. *PrimeSearcher.do* is just a name given to our controller servlet and *.do* is just a convention. The *GET* method indicates that whenever a client clicks the *GENERATE PRIME* button, the *doGet* method of the servlet is called.

The deployment descriptor file, *web.xml* looks like what's given below:

```

<web-app xmlns="http://java.sun.com/xml/ns/j2ee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/j2ee/web-app_2_4.xsd"
  version="2.4">
  <servlet>

    <servlet-name>PrimeSearcher</servlet-name>
    <servlet-class>Controller</servlet-class>
  </servlet>

  <servlet-mapping>
    <servlet-name>PrimeSearcher</servlet-name>
  
```